NORTH BENGAL INTERNATIONAL UNIVERSITY

Lab Report : 5

TOPIC : WRITE THE PROGRAMS OF LECTURE-6, HOMEWORKS & A PROGRAM TO SEE MY CGPA

COURSE CODE : CSE 0612-2122

COURSE TITLE : OBJECT ORIENTED PROGRAMMING SESSIONAL.

Submitted By:

MD. TANVIR RAHMAN PRANTO

0712340012101005

CSE -26

Submitted To:

AMIT KUMAR PAUL

LECTURER, CSE

Write a program of lecture-6, Homework & a program to see my cgpa:

Single inheritance:

Code:

```
#include <iostream>
using namespace std;

// Base class
class Person {
public:
    void display() {
        cout << "This is a person." << endl;
    }
};

// Derived class
class Student : public Person {
public:
    void study() {
        cout << "The student is studying." << endl;
    }
};

int main() {
    Student myStudent;

    myStudent.display();

    myStudent.study();

    return 0;
}
```

Output:

```
OUTPUT    TERMINAL

This is a person.
The student is studying.
PS C:\Users\Lenovo\Documents\c++>
```

Multilevel inheritance:

Code:

```
#include <iostream>
using namespace std;
// Base class
class Device {

public:
    void powerOn() {
```

```
        cout << "Device powered on." << endl;
    }
};

// Derived class from Device
class Computer : public Device {
public:
    void boot() {
        cout << "Windows booting up." << endl;
    }
};

// Derived class from Computer
class Laptop : public Computer {
public:
    void ssd() {
        cout << "This Laptop has a fast ssd." << endl;
    }
};

int main() {
    Laptop myLaptop;

    myLaptop.powerOn();

    myLaptop.boot();

    myLaptop.ssd();

    return 0;
}
```

Output:

```
OUTPUT    TERMINAL

Device powered on.
Windows booting up.
This Laptop has a fast ssd.
PS C:\Users\Lenovo\Documents\c++>
```

Multiple inheritance:

Code:

```
#include <iostream>
using namespace std;

class Printer {
public:
    void print() {
        cout << "Printing document." << endl;
    }
};
```

```
class Scanner {
public:
    void scan() {
        cout << "Scanning document." << endl;
    }
};

// Derived class
class AllInOne : public Printer, public Scanner {
public:
    void fax() {
        cout << "Sending fax." << endl;
    }
};

int main() {
    AllInOne myDevice;

    myDevice.print();
    myDevice.scan();

    myDevice.fax();

    return 0;
}
```

Output:

```
OUTPUT    TERMINAL

Printing document.
Scanning document.
Sending fax.
PS C:\Users\Lenovo\Documents\c++>
```

Multiple inheritance:

Code:

```
#include <iostream>
using namespace std;

// Base class 1
class Person {
public:
    void walk() {
        cout << "Person is walking." << endl;
    }
};

// Base class 2
class Worker {
public:
```

```
    void work() {
        cout << "Worker is working." << endl;
    }
};

// Derived class inheriting from both Person and Worker
class Engineer : public Person, public Worker {
public:
    void design() {
        cout << "Engineer is designing." << endl;
    }
};

// Another base class
class Artist {
public:
    void draw() {
        cout << "Artist is drawing." << endl;
    }
};

// Derived class inheriting from Engineer and Artist
class Architect : public Engineer, public Artist {
public:
    void build() {
        cout << "Architect is building." << endl;
    }
};

int main() {
    Architect myArchitect;

    // Calling base class functions
    myArchitect.walk();
    myArchitect.work();
    myArchitect.design();
    myArchitect.draw();
    myArchitect.build();

    return 0;
}
```

Output:

```
OUTPUT    TERMINAL

Person is walking.
Worker is working.
Engineer is designing.
Artist is drawing.
Architect is building.
PS C:\Users\Lenovo\Documents\c++>
```

Hierarchical inheritance:

Code:

```
#include <iostream>
using namespace std;

// Base class
class Shape {
public:
    void draw() {
        cout << "Drawing a shape." << endl;
    }
};

// Derived class 1
class Circle : public Shape {
public:
    void draw() {
        cout << "Drawing a circle." << endl;
    }
};

// Derived class 2
class Rectangle : public Shape {
public:
    void draw() {
        cout << "Drawing a rectangle." << endl;
    }
};

// Derived class 3
class Triangle : public Shape {
public:
    void draw() {
        cout << "Drawing a triangle." << endl;
    }
};

int main() {
    Circle myCircle;
    Rectangle myRectangle;
    Triangle myTriangle;

    // Calling base class function through derived classes
    myCircle.draw();
    myRectangle.draw();
    myTriangle.draw();

    return 0;
}
```

Output:

```
OUTPUT    TERMINAL

Drawing a circle.
Drawing a rectangle.
Drawing a triangle.
PS C:\Users\Lenovo\Documents\c++>
```

Polymorphism:

Code:

```
#include <iostream>
using namespace std;

// Base class
class Employee {
public:
    virtual void work() {
        cout << "Employee is working." << endl;
    }
};

// Derived class 1
class Manager : public Employee {
public:
    void work() override {
        cout << "Manager is managing." << endl;
    }
};

// Derived class 2
class Engineer : public Employee {
public:
    void work() override {
        cout << "Engineer is developing." << endl;
    }
};

// Derived class 3
class Intern : public Employee {
public:
    void work() override {
        cout << "Intern is learning." << endl;
    }
};

void showWork(Employee &employee) {
    employee.work();
}

int main() {
    Manager myManager;
    Engineer myEngineer;
    Intern myIntern;

    // Using a function that demonstrates polymorphism
    showWork(myManager);
    showWork(myEngineer);
    showWork(myIntern);

    return 0;
}
```

Output:

```
OUTPUT    TERMINAL

Manager is managing.
Engineer is developing.
Intern is learning.
PS C:\Users\Lenovo\Documents\c++>
```

Encapsulation:

Code:

```
#include <iostream>
using namespace std;

class Student {
private:
    // Private data members
    string name;
    int rollNumber;

public:
    // Constructor
    Student(string studentName, int studentRollNumber) {
        name = studentName;
        rollNumber = studentRollNumber;
    }

    // Public method to get student's name
    string getName() const {
        return name;
    }
    // Public method to get student's roll number
    int getRollNumber() const {
        return rollNumber;
    }
};

int main() {
    // Create a Student object
    Student myStudent("md.pranto", 005);

    cout << "Student Name: " << myStudent.getName() << endl;
    cout << "Student Roll Number: " << myStudent.getRollNumber() <<
endl;

    return 0;
}
```

Output:

```
OUTPUT    TERMINAL

Student Name: md.pranto
Student id: 5
PS C:\Users\Lenovo\Documents\c++>
```

Cgpa calculation:

Code:

```
#include <iostream>
using namespace std;

int main() {
    int numberOfCourses;
    cout << "Enter the total number of courses: ";
    cin >> numberOfCourses;

    int totalCredits = 0;
    double totalGradePoints = 0;

    for (int i = 0; i < numberOfCourses; ++i) {
        int credits;
        double gradePoint;
        cout << "Enter credits for course " << i + 1 << ": ";
        cin >> credits;
        cout << "Enter grade point for course " << i + 1 << ": ";
        cin >> gradePoint;

        totalCredits += credits;
        totalGradePoints += gradePoint * credits;
    }

    double cgpa = totalGradePoints / totalCredits;
    cout << "The CGPA is: " << cgpa << endl;

    return 0;
}
```

Output:

```
OUTPUT    TERMINAL

Enter the total number of courses: 3
Enter credits for course 1: 3
Enter grade point for course 1: 3.6
Enter credits for course 2: 4
Enter grade point for course 2: 3.8
Enter credits for course 3: 2
Enter grade point for course 3: 3.2
The CGPA is: 3.6
PS C:\Users\Lenovo\Documents\c++>
```